

```

while(check_location >= 0):
    stepcounter += 1
    if to_insert > cabinet[check_location]:
        insert_location = check_location + 1
        check_location = - 1
    check_location = check_location - 1
    stepcounter += 1
cabinet.insert(insert_location,to_insert)
return(cabinet)

def insertion_sort(cabinet):
    newcabinet = []
    global stepcounter
    while len(cabinet) > 0:
        stepcounter += 1
        to_insert = cabinet.pop(0)
        newcabinet = insert_cabinet(newcabinet,to_insert)
    return(newcabinet)

cabinet = [8,4,6,1,2,5,3,7]
stepcounter = 0
sortedcabinet = insertion_sort(cabinet)
print(stepcounter)

```

Listing 4-3: *Our insertion sort code with a step counter*

In this case, we can run this code and see that it performs 36 steps in order to accomplish the insertion sort for a list of length 8. Let's try to perform insertion sort for lists of other lengths and see how many steps we take.

To do so, let's write a function that can check the number of steps required for insertion sort for unsorted lists of different lengths. Instead of manually writing out each unsorted list, we can use a simple list comprehension in Python to generate a random list of any specified length. We can import Python's `random` module to make the random creation of lists easier. Here's how we can create a random unsorted cabinet of length 10:

```

import random
size_of_cabinet = 10
cabinet = [int(1000 * random.random()) for i in range(size_of_cabinet)]

```
